

Bismuth: Protocol and Node Architecture

Version: 2.0

Date: 2026-06-28

www.bismuth.cz

Abstract

Bismuth is a proof-of-work blockchain whose `hf2` activation unified its major protocol upgrades into one coherent execution model. In its active form, the network combines a memory-hard mining algorithm, account-based transaction semantics, content-addressed transaction identity, deterministic binary and integer serialization, a signer-agnostic verification framework, a committed RISC-V virtual machine, optional shielded value transfer, and a storage architecture built around canonical chain data plus rebuildable key-value projections.

The protocol is implemented by a full Python node that remains deliberately inspectable. Consensus-critical behavior is centralized in a small set of modules that define signing preimages, block hashing, difficulty selection, proof-of-work validation, spendability, replay protection, and post-fork state commitments. Around that core, the node exposes both a legacy socket protocol and a parallel REST interface, while moving heavy derived state such as contract storage, shielded notes, token and alias indexes, balances, and transaction identifiers into rollback-safe key-value stores. The result is a system that preserves a simple base transaction model while supporting a much richer protocol surface than a conventional single-purpose payment chain.

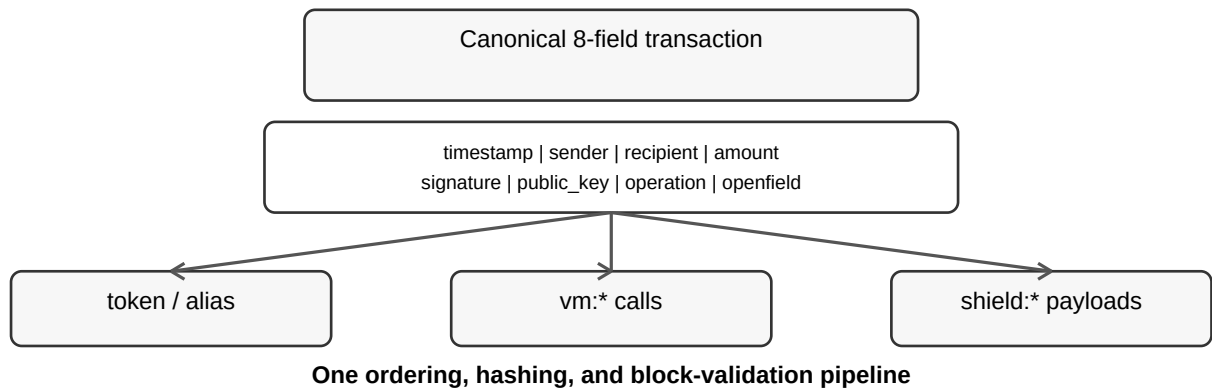
1. Introduction

The protocol is an account-based blockchain in which the base ledger remains intentionally generic. A transaction is represented as an eight-field tuple containing a timestamp, sender, recipient, amount, signature, public key, operation string, and open field. Rather than fragmenting the chain into many distinct transaction families, the network uses the `operation` and `openfield` pair as a narrow semantic extension channel. Tokens, aliases, virtual-machine calls, shielded transfers, and application-specific protocols all ride on the same base ledger object and are therefore ordered, hashed, propagated, and archived through one consensus path.

The `hf2` fork is the architectural boundary that turned this design from a historically layered codebase into a coherent modern protocol. After activation, Bismuth no longer identifies a transaction by slicing the signature, no longer relies on a legacy string representation for new consensus hashes, no longer places the mining nonce in the coinbase openfield, and no longer treats contract-state integrity as an off-chain convention. Instead, `hf2` activated content-derived transaction identifiers, recoverable compact secp256k1 signatures for ordinary single-signature senders, binary and integer block hashing, a blake2b variant of Heavy3, LWMA difficulty retargeting, coinbase compaction with a dedicated mining header, coinbase reward maturity, VM state-root commitments, and the fork-gated validation paths required by native multisig senders and shielded-value operations.

The resulting network is best understood as a modular proof-of-work settlement layer. Its consensus engine remains conservative and explicit, but the interpretation layers above it are broad enough to host private transfers, decentralized exchange components, contract-based games, multisignature custody, and browser-driven applications without forcing those higher-order features to redefine the base ledger format.

The relationship between the base transaction and the higher-order protocol layers can be summarized as follows:



2. Design Principles

Bismuth follows a small number of protocol principles very consistently. First, consensus bytes are frozen explicitly. Signing buffers, transaction identifiers, and block-hash preimages are not allowed to drift implicitly when storage, transport, or wallet code changes. They are defined centrally and guarded by characterization tests, so a refactor in an API handler or a database codec cannot silently fork the chain. Second, the project prefers one fork boundary for coupled changes rather than many independent toggles. The `hf2` activation height therefore gates a coordinated family of serialization, signature, PoW, difficulty, and execution changes that would be unsafe if activated piecemeal.

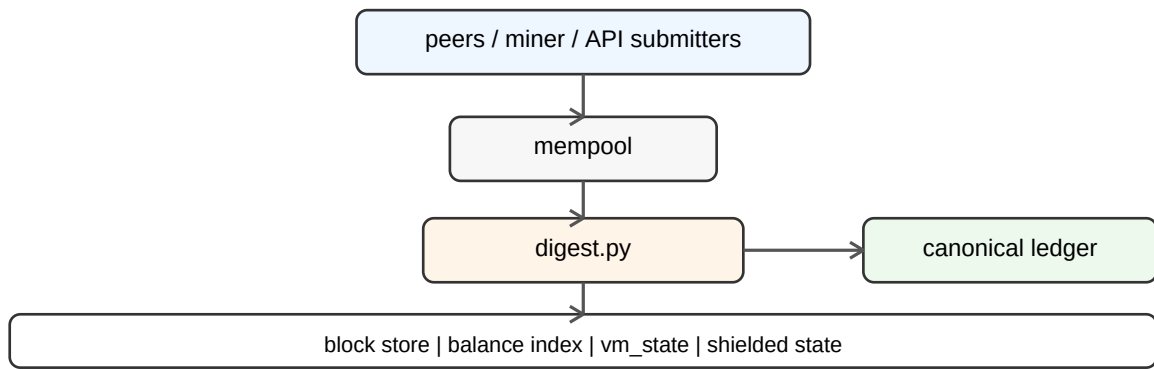
Third, the node separates canonical chain data from rebuildable projections. The ledger determines the history of the network, but balances, transaction-id indexes, contract storage, shielded pools, token registries, alias ownership, and explorer statistics are treated as materialized views that must be re-creatable from the chain and removable on rollback. Fourth, the node modernizes around the edges rather than rewriting the core blindly. REST transport, key-value storage seams, replay verification, parity checks, and block-store migration all exist to reduce operational and implementation complexity without changing what the chain means. Fifth, the project treats deterministic execution as the non-negotiable condition for any richer functionality. Contract execution, shielded-state transitions, and post-fork signature dispatch all operate under this constraint.

3. System Overview

The reference node is a single multithreaded Python process. That process runs the consensus engine, peer-to-peer server, mempool, mining validator, optional solo miner, legacy socket API, and optional REST API in one address space. This architecture is not accidental. It keeps the validation path observable end to end and avoids the need for a separate message bus or distributed coordination layer inside the node itself.

The node serves mainnet traffic on TCP port `5658`, testnet on `2829`, and regnet on `3030`. When the REST API is enabled, it typically serves on `5659`. Inbound legacy socket connections are handled by a threaded TCP server. Outbound worker threads connect to peers, exchange version and synchronization commands, fetch candidate blocks, and pass them into the digester. A connection manager thread handles peer maintenance, mempool cleanup, and periodic state logging. When enabled, the REST server handles HTTP requests concurrently in their own threads, each with short-lived database access. Auxiliary loops, such as difficulty-divergence detection and self-heal logic, run independently and never redefine block validity on their own.

This single-process design gives the system a sharply defined consensus shell. Candidate blocks enter through peer or local-miner paths, but they are all reduced to the same `digest\_block` admission route. Mempool transactions may arrive over the legacy socket protocol or over the optional REST write endpoint, but they all pass through the same `mempool.merge` validation path before they can be mined.



4. Consensus Pipeline

The consensus core lives in `digest.py`, with transaction parsing and value normalization in `digest_tx.py`. Every block admitted to the chain passes through a strictly ordered sequence. The node first rejects obviously hostile or banned peers, then acquires the ledger lock, waits for mempool safety, parses each transaction, checks signature validity, enforces timestamp and replay rules, recomputes the current target difficulty, rebuilds the block hash from the canonical post-fork representation, validates Heavy3 proof of work, enforces balance and fee rules, applies post-fork feature gates such as VM state-root checks and shielded-block validation, commits the block atomically, removes confirmed mempool entries, updates checkpoints, and finally advances auxiliary indexes and hooks.

This ordering matters. In Bismuth, proof of work is validated only after the transaction list has been normalized into its consensus representation, because the block hash and the mining target are both defined over that converted list. Balance enforcement is applied before a block is committed, but after all transactions in the block are known, because Bismuth validates the cumulative debit and fee burden an address incurs within the candidate block rather than only checking transaction by transaction in isolation. Similarly, post-fork VM and shielded validation occurs before commit, but after the block content, miner header, and reward semantics have already been made canonical.

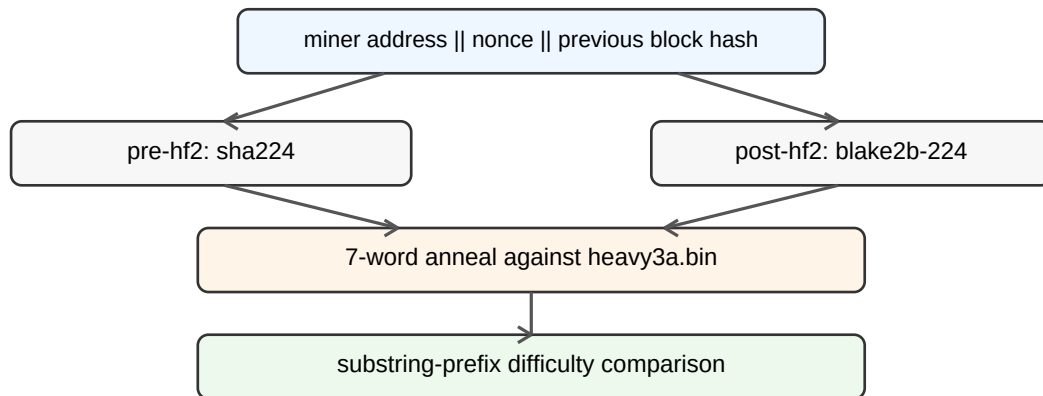
The node maintains explicit rollback and recovery behavior around this pipeline. If validation fails at any stage, the block is rejected, the peer is penalized, and the tip is restored from authoritative state. If a rollback occurs, the chain is rewound and every registered auxiliary store must also be rewound or rebuilt to the same height. This discipline is one of the defining qualities of the codebase.

5. Heavy3 Proof of Work

The network uses Heavy3, a memory-hard proof-of-work function whose cost depends not only on hashing but also on random access into a deterministic 1 GiB lookup file called `heavy3a.bin`. The file is generated from an HMAC-DRBG seeded with a fixed ASCII description of bismuth the element, written in fixed-size chunks, and validated against sentinel values on load. Because the file is deterministic, every node can regenerate it if needed. Because it is large and randomly addressed, it serves as the memory-hard component of the mining function.

The mining preimage is based on the miner address, a nonce, and the previous block hash. That preimage is hashed into a 224-bit value, which is then annealed by XOR-ing its seven 32-bit words against seven pseudo-randomly selected words from the 1 GiB table. The resulting value is interpreted through a substring-prefix difficulty measure: the chain difficulty is the longest prefix of the previous block hash's bit representation that can be found inside the annealed bitstring. This unusual measure is fixed consensus behavior, and every validator and miner derives it in the same way.

After `hf2`, the protocol continues to use the same Heavy3 structure but changes the inner hash from `sha224` to `blake2b` with a 28-byte output. The lookup file, annealing step, and substring difficulty logic are unchanged. In other words, the fork modernizes the digest primitive without changing the memory-hard geometry of the work itself. This makes the upgrade operationally clear: below the activation height the chain validates and mines legacy Heavy3, and from the activation height onward it validates and mines the blake2b form.



6. Difficulty Retargeting

The chain targets a 60-second block interval. Before `hf2`, the node used a PID-style difficulty controller that estimated hashrate from recent solve times, applied a derivative feedback term, damped the change, and then imposed a one-sided cap on upward adjustment. That controller worked, but it behaved poorly on a small and bursty network because it could move up quickly and drift down only slowly, especially when transient hashpower entered and then vanished.

The `hf2` protocol replaces the live output of that controller with an LWMA retarget. LWMA computes a linearly weighted moving average of recent solve times, giving more influence to recent blocks while still smoothing short-term noise. Its adjustment law is symmetric in the logarithmic work domain, which means the chain responds to sustained fast blocks and sustained slow blocks under the same rule. The implementation clamps individual solve times to resist timestamp manipulation, computes entirely in fixed-precision Decimal arithmetic for cross-platform determinism, and applies a bounded per-step change. This is a simpler and safer rule for a small proof-of-work network.

The implementation also contains a node-level defense against corrupted local difficulty state. Because the historical controller referenced cached previous difficulty values from persistent storage, a damaged local view could become self-reinforcing. The node therefore includes a background divergence detector that polls height-matched peers through REST, compares the local difficulty view against the peer median, debounces the result over multiple cycles, and can schedule a rollback-and-resync if the local node appears trapped on a wrong difficulty track. This mechanism does not alter consensus. It is an operational healing path for a broken local node.

7. Transactions and Signatures

The ledger is account-based. A normal transaction names a sender address, a recipient address, an amount, a signature, a public key or key reference, and optional semantic payload fields. The sender balance is authoritative, and the digester enforces not only that the sender can cover the amount but also that the sender can cover the sum of all block-local debits and fees attributed to it in the candidate block.

The protocol routes signature handling through a signer abstraction called `polysign`. This abstraction supports RSA, secp256k1 ECDSA, ED25519, native multisig, ML-DSA families, and secp256r1. The

important consequence is that the protocol does not hardcode a single signature family into the consensus engine. Instead, the sender address determines the signer class, and verification dispatches accordingly. This is one reason the architecture is structurally well positioned for future cryptographic pivots.

After `hf2`, ordinary single-signature secp256k1 transactions use a recoverable-signature model. The sender signs the canonical transaction content hash rather than a raw tuple buffer, the signature is stored as a 65-byte recoverable compact encoding, the public key is omitted from the transaction, and the validator reconstructs the signing key from the signature and message and then derives the sender address from that recovered key. The transaction therefore commits to content directly, and its identity no longer depends on the incidental shape of a DER or base64-encoded signature. Low-s enforcement makes this representation canonical.

Other signature families remain explicit where recovery is impossible or undesirable. ED25519 keys can be reconstructed from their address family, while RSA, ML-DSA, and native multisig may rely on explicit or by-reference public-key resolution. The unifying principle is that the chain always validates against one authoritative content preimage, but the transport representation of the key material may differ by signer family.

8. Serialization and Transaction Identity

One of the most important `hf2` changes is the move away from legacy string-serialized consensus objects. Historically, Bismuth used the UTF-8 `repr` of a six-field tuple as the signing buffer and the `repr` of a list of eight-field converted transactions concatenated with the previous block hash as the block-hash preimage. Those byte forms remain frozen for pre-fork history, but `hf2` activates a different post-fork representation.

The post-fork transaction identifier is a content hash. It is derived from a canonical preimage that excludes the signature and public key and is hashed with `blake2b-256`. The block hash likewise moves to a binary and integer representation in which timestamps are stored as integer centiseconds, amounts are stored as integer atomic units, variable-length fields are explicitly length-prefixed, and the whole block body is hashed through `blake2b-256`. This removes ambiguity, eliminates dependencies on Python tuple stringification for new consensus objects, and allows storage layers to change internally without redefining transaction or block identity.

The chain still maintains precise continuity across the fork boundary. The dispatcher chooses the legacy or post-fork byte form by destination block height. As a result, a historical block remains byte-identical when replayed, while a post-fork block always hashes under the new rules. This height-based dispatch is one of the reasons the migration is safe. There is no global "serialization mode" switch that could accidentally reinterpret old history.

9. Coinbase Redesign and Reward Maturity

The coinbase transaction in the `hf2` era is not treated as a normal signed spend. It is authorized by proof of work and reward rules, not by a sender signature. Bismuth therefore repurposes the freed `signature` and `public\_key` slots of the coinbase as a mining header. The `signature` field carries the proof-of-work nonce, and the `public\_key` field carries the mining commitment, which includes the VM state-root commitment and optionally the fork-readiness signal when needed. Because these fields are committed into the post-fork block-hash preimage, the miner cannot alter them without invalidating the block.

This design also frees the coinbase `operation` and `openfield` fields from their former role as overloaded transport for nonce, signal, and state-root data. They become optional miner data fields rather than mandatory protocol containers. The resulting coinbase is both smaller and cleaner than the pre-fork construction.

`hf2` also activates coinbase maturity. A newly mined reward cannot be spent immediately. Instead, Bismuth imposes a fixed maturity window of 100 blocks and deducts immature reward balance from the miner's spendable balance when validating spends. In an account-based system this is an important safety rule

because there is no isolated UTXO output to lock. Without maturity, a deep enough reorg could retroactively invalidate downstream payments funded by a fresh reward. Maturity prevents that by making the reward temporarily non-spendable even though it is already visible in the ledger.

10. Fee Model

The protocol retains a simple fee structure but makes it more responsive after `hf2`. Before the fork, fees were based on a static base fee plus payload-dependent surcharges. After activation, the node derives a dynamic base fee from recent block weight recorded in the block store. Weight is not merely a transaction count. It reflects the structural load imposed by a block, including openfield payload size. This matters because the chain carries heterogeneous transaction classes whose validation and storage burden can vary significantly.

The resulting model is closer to a deterministic congestion-sensitive floor than to a general-purpose gas market. A base multiplier is derived from recent observed block weight relative to a target, clamped within a bounded range, and applied to the static baseline. Additional surcharges can then apply to feature classes such as VM or shielded operations whose validation burden is structurally higher than that of a simple payment.

11. Ledger and Storage Architecture

Historically, the node used several SQLite databases, notably `ledger.db`, `hyper.db`, `index.db`, and `mempool.db`. That architecture is still visible in the reference node, but the `hf2` era is fundamentally defined by a movement away from "one SQL table for everything" and toward one canonical chain plus rebuildable projections. The canonical history remains the chain itself. Around it, the node now maintains specialized stores for block bodies, balances, transaction identifiers, contract state, shielded notes, reward accounting, token balances, and alias ownership.

The `block\_store` is the keystone of this direction. It stores block bodies by height in an append-only key-value structure and indexes block hashes back to heights. Because the block store preserves the exact consensus rows needed to reconstruct post-fork transactions, it can serve as a canonical or near-canonical body layer independent of any SQL representation. Around it sit key-value projections such as `balance\_index`, `txid\_index`, `reward\_chain`, `vm\_state`, `ShieldedState`, and `token\_index`.

The project has taken care not to trust these stores blindly. Storage migration is routed through an engine-agnostic seam in `kvstore.py`, which lets the same logical stores run on LMDB, and potentially other key-value engines, without rewriting each subsystem. Just as importantly, cross-checking and parity assertion are part of the design. Derived balance reads, duplicate-replay reads, and block-body reads are subjected to shadow and primary modes in tests and staging. Replay verification computes canonical block hashes again from chain data. Auxiliary states are rebuilt after reorgs. The governing principle is simple: a new store is allowed to accelerate or simplify the node only after it proves that it means exactly the same thing as the old path.

12. Networking

The native peer-to-peer protocol is a raw TCP protocol that frames each message as a JSON payload preceded by a ten-byte zero-padded ASCII length header. This protocol is intentionally simple. Peers exchange version information, peer lists, synchronization triggers, block heights, mempool content, and freshly mined blocks over this channel. The protocol remains the baseline network compatibility layer even though the node also exposes richer HTTP services.

Peer management is handled by a mixin-based `Peers` object that persists peer sets, limits retries, tracks bans and warnings, records peer opinions about the tip height, and maintains a local reputation score for peers that have delivered valid or invalid data. Reputation does not override validation, but it does harden the selection

of which peer view to trust when the node must decide how to recover from a stale or disputed tip.

The node also exposes a modern REST API. That API does not replace consensus and does not create a second validation path; it exposes the same chain through a more parallel transport model. Nodes can serve status information, difficulty state, balances, block ranges, headers, peers, fee information, VM contract state, shielded-pool metrics, token and alias data, and optional transaction-submission endpoints over HTTP. REST also serves as the capability-discovery layer for newer synchronization patterns, including headers-first and range-based block fetches.

In addition to live sync, Bismuth supports peer-to-peer snapshots. A node can serve a prebuilt ledger snapshot and a manifest that states the height, size, and checksum. Another node can fetch that snapshot from peers, verify its checksum, install it, and then continue normal consensus validation from the snapshot height forward. This reduces reliance on a single centralized bootstrap host while preserving full verification from the snapshot boundary onward.

13. Mempool and Anti-Abuse Logic

The mempool is deliberately narrow in responsibility. It is a cache of candidate transactions that are already close to consensus-valid, not an alternative settlement layer. Transactions are accepted only after signature checks, duplicate checks, balance checks, structural bounds checks, and fee-policy checks. For post-fork rules such as coinbase maturity and signer-family dispatch, the mempool mirrors the logic used by the digester so that obviously invalid transactions can be rejected before they ever burden miners or validators further downstream.

Bismuth also treats protocol hardening as part of node design. Field lengths are bounded. Transaction timestamps are restricted to plausible windows. Duplicate checks are height-aware. Freshly matured or immature balances are treated explicitly. Peer misbehavior accumulates warnings and can produce bans. REST relay functionality is guarded against SSRF, excessive concurrency, and rate abuse. These constraints are not ornamental; they are part of how the node stays operable under adversarial network conditions.

14. Virtual Machine

Bismuth's contract layer is a deterministic RISC-V RV32I virtual machine implemented in ``bismuth_riscv.py``, orchestrated by ``vm_engine.py``, and backed by ``vm_state.py``. The design rejects the idea that every richer application requires a bespoke execution model. Instead, Bismuth chooses one deterministic instruction set and one deterministic state-transition engine, then lifts higher-order applications into contracts that compile or assemble to that machine.

Contracts are introduced through ``vm:deploy`` transactions and invoked through ``vm:call`` transactions. The contract address is derived from the deploy transaction's canonical content txid rather than from its signature. This is important because it makes contract identity depend on transaction content, not on any malleable or signer-family-specific encoding choice.

The most important aspect of the VM is that it can custody and move real BIS. A contract does not merely read ledger state and emit advisory messages. Value sent to a contract is transferred into a dedicated custody sink on the ledger and then mirrored into a contract balance inside ``vm_state``. When a contract transfers value outward, the node debits the contract's custody balance and writes deterministic payout rows from the custody sink to the recipient. Because those custody balances live in the contract state store rather than being inferred from arbitrary historical ledger balance, a rollback can rebuild them exactly by replaying the contract calls. This is the core determinism property that makes value-bearing contracts viable.

``hf2`` also makes the VM state root a consensus object. The miner commits the pre-state root into the coinbase mining header, and the digester rejects a block if its own locally derived state root does not match the committed one. The state root covers contract code, storage slots, and custody balances. As a result, a VM implementation bug becomes a visible fork event rather than a silent long-term drift.

The repository already includes reference contracts and applications for escrow, vesting, raffles, prediction markets, tokens, decentralized exchange components, AMMs, routers, poker, and contract-based multisignature custody. These are not all consensus rules, but they demonstrate the intended use of the VM as a general application substrate.

15. Shielded Value

The protocol's shielded layer is a consensus-validated privacy pool carried inside ordinary base transactions. Its purpose is to replace the earlier off-chain shielded prototype, whose balances and spent-set logic lived outside consensus and therefore could not provide settlement-grade guarantees. In the `hf2` era, privacy operations are validated by every full node through the same block-admission process that validates fees, signatures, and proof of work.

The shielded design is staged but unified in one module. At the first stage, it provides stealth-address output construction so that only the intended recipient can recognize and spend a note, while outside observers cannot link the note directly to the recipient's published shielded address. At the second stage, spends use linkable ring signatures and key images so that the chain can prevent double spends without revealing which member of a ring was actually spent. At the third stage, amounts can be hidden through RingCT commitments and Bulletproof range proofs.

Importantly, shielded transfers do not create a second blockchain inside Bismuth. They are ordinary Bismuth transactions with `shield:\*` operations whose payloads define notes, ring members, key images, commitments, and redeems. Their transparent envelope gives them a place in total order, while their inner cryptographic content determines private ownership and spend validity. The shielded state itself is stored in a dedicated key-value sidecar that tracks notes, height-indexed rollback entries, key images, and flow accounting, all of which can be rewound or rebuilt to the canonical chain tip.

16. Wallets, HD Keys, Multisig, and Post-Quantum Readiness

The system supports multiple signer families, but it also supports multiple wallet-management models. Legacy RSA wallets remain supported, while secp256k1 ECDSA and ED25519 addresses serve modern user flows. HD wallets based on BIP32-style derivation and BIP39 mnemonics allow a wallet to derive many addresses from one seed without any consensus change. This works naturally because the chain only cares about addresses and signatures, not about how a private key was derived.

Native multisignature support exists at the base-layer sender level as well as at the VM level. In the native case, a multisig address is derived from a threshold and a sorted set of owners, the redeem information rides in the transaction, and the node verifies that a sufficient threshold of distinct owner signatures authorizes the spend. At the VM level, a multisig vault is simply a contract whose state machine expresses approval and execution policy. The existence of both models is useful. Native multisig is leaner and more directly tied to signer semantics, while VM multisig is more programmable.

The architecture is also structurally prepared for a future post-quantum pivot. The repository already contains ML-DSA signer implementations and tests, and the signer-factory design means that activating a new signature family is principally a consensus-dispatch and wire-format question rather than a total redesign of how the node thinks about identities. That does not mean the chain has already made a PQ transition, only that the groundwork is present and far lighter than it would be in a protocol welded permanently to one classical signature family.

17. Tokens, Aliases, and Plugins

One of the protocol's long-term architectural themes is the separation of base consensus from derived interpretation layers. Tokens and aliases illustrate this well. They are built from ordinary ledger transactions, but their higher-level meaning is maintained in dedicated indexes rather than baked into the block-hash or

proof-of-work path. In the modern node, those indexes can live behind a plugin framework that owns its own storage, backfill logic, rollback hooks, wire-command handlers, and REST routes.

This approach keeps the consensus core smaller while still enabling a richer ecosystem of naming, asset, and application semantics. The base chain remains the source of truth, but its meaning can be extended in a modular way where block validity is not affected.

18. Security Model

The security model can be summarized in four claims. First, every consensus-critical transition must be a pure function of on-chain data plus explicit fork height. This is why serialization dispatch, signer-family verification, LWMA, shielded validation, and VM state-root enforcement all live in narrow, deterministic code paths. Second, the node assumes that crashes, partial writes, and reorgs are normal hazards rather than exceptional events. Recovery and rollback therefore have to be designed into every auxiliary store, not added later.

Third, storage modernization is never allowed to redefine meaning. New projections can become authoritative only after they survive replay verification and parity checking. Fourth, the protocol distinguishes carefully between network convenience and consensus legitimacy. REST sync, snapshot fetches, proxy relays, explorer statistics, and compatibility RPCs can improve usability and operator experience, but none of them are allowed to create an alternate route around the consensus engine.

19. Conclusion

Bismuth in the `hf2` era is a proof-of-work blockchain that deliberately combines conservative consensus engineering with broad protocol expressiveness. It retains a simple account-based base ledger, but that base ledger now supports content-addressed transaction identity, canonical binary hashing, recoverable compact signatures, mature key-value state models, deterministic contract execution with committed state roots, consensus-enforced shielded transfers, native and contract-based multisig, and a transport stack that ranges from a minimal raw TCP protocol to a concurrent REST API.

What makes Bismuth technically distinctive is not any one of those features by itself. It is the way they are organized. The chain keeps one canonical ordering engine, one proof-of-work family, one explicit fork boundary for tightly coupled upgrades, and one persistent insistence that storage, transport, and application layers must not silently rewrite consensus. That discipline is what allows the network to grow more capable without becoming structurally ambiguous.

Appendix A: Principal Modules

The principal consensus modules are ``digest.py``, ``digest_tx.py``, ``fork.py``, ``difficulty.py``, ``difficulty_lwma.py``, ``mining_heavy3.py``, and ``bismuth_serialize.py``. The principal storage and index modules are ``dbhandler.py``, ``block_store.py``, ``storage_backend.py``, ``kvstore.py``, ``balance_index.py``, ``txid_index.py``, ``reward_chain.py``, and ``token_index.py``. The main networking and API modules are ``node.py``, ``worker.py``, ``connections.py``, ``peershandler.py``, ``rest_api.py``, ``rest_client.py``, and ``api_sync.py``. The execution and privacy layers are centered on ``bismuth_riscv.py``, ``vm_engine.py``, ``vm_state.py``, ``shieldedv1.py``, ``ringct.py``, and ``bulletproof.py``. Wallet and signer logic is concentrated in ``wallet_helpers.py``, ``hd_wallet.py``, ``multisig_wallet.py``, and the signer classes under ``polysign/``.

Appendix B: Source Basis

This document was derived from the implementation and the protocol notes in this repository, especially ``doc/00-architecture-overview.md``, ``doc/03-consensus-blocks-digest.md``, ``doc/04-pow-and-difficulty.md``, ``doc/06-networking-protocol.md``, ``doc/15-rest-api.md``, ``doc/18-hardfork-hf2.md``, ``doc/19-vm.md``, ``doc/20-post-quantum.md``, ``doc/21-mining.md``, ``doc/22-shielded.md``, ``doc/23-hd-multisig.md``, ``doc/26-storage-postfork.md``, ``doc/27-plugins.md``, ``doc/29-hf2-serialization-v2.md``, ``doc/35-difficulty-divergence-selfheal.md``, ``doc/41-hf2-coinbase-free-fields.md``, ``doc/42-coinbase-maturity.md``, and ``doc/43-p2p-snapshot.md``.